

Capitol Ledger Backend Setup Recommendations

Reference guide for outside services needed to move Capitol Ledger from demo mode toward production. Updated May 22, 2026.

Recommended Starter Stack

Vercel + Neon Postgres + Resend + Stripe + Vercel Cron + Upstash Redis + OpenAI API + Congress.gov API + Vercel Observability.

Best Starter Stack

Backend Need	Recommendation	Why
App hosting	Vercel	Best fit for Next.js, environment variables, preview deployments, logs, and cron jobs.
Database	Neon Postgres	Managed Postgres that works well with Prisma. Branching helps test migrations safely.
Database alternative	Supabase Postgres	Good dashboard and backup tooling if you want a broader backend platform.
Auth email	Resend	Simple developer email API for verification, reset links, and future briefs.
Email alternative	Postmark	Strong transactional reliability if deliverability becomes the top priority.
Subscriptions	Stripe	Already matches Capitol Ledger checkout and webhook architecture.
Weekly Brief schedule	Vercel Cron	Simplest if hosting the app on Vercel.
Rate limiting	Upstash Redis	Good fit for auth, password reset, checkout, AI, and public API abuse protection.
AI analysis	OpenAI API	Best fit for bill summaries, policy lens, and plain-language pros/cons.
Government data	Congress.gov API	Official source for federal bills, members, committees, and summaries.
Monitoring	Vercel Observability first	Start simple. Add Sentry later if errors need deeper tracing.

Recommended Setup Order

- 1 Create the hosted Postgres database, then run the checked-in Prisma migration and ``pnpm production-auth:check``.
- 2 Deploy the app with HTTPS and configure core environment variables: ``NEXT_PUBLIC_APP_URL``, ``AUTH_SECRET``, and ``AUTH_COOKIE_SECURE=true``.
- 3 Connect auth email through the existing webhook bridge and test verification plus password reset flows.
- 4 Add the Congress.gov API key and schedule live data sync after the database is ready.
- 5 Create Stripe products and prices for Pro monthly, Pro annual, Civic Team monthly, and Civic Team annual.
- 6 Configure the Stripe webhook endpoint at ``/api/billing/stripe/webhook``, then run ``BILLING_REQUIRE_STRIPE=true pnpm billing:check``.
- 7 Connect Weekly Brief delivery through the webhook bridge, add the scheduler, and run ``pnpm weekly-brief:check`` plus ``pnpm weekly-brief:qa``.
- 8 Add Upstash Redis for persistent rate limiting before heavier public traffic.
- 9 Add OpenAI API once the live bill record/source pipeline is ready for generated summaries and policy analysis.

- 10 Add deeper monitoring, backups, and restore drills before launch.

Environment Checklist

Area	Important variables
Core app	<code>`DATABASE_URL`, `NEXT_PUBLIC_APP_URL`, `AUTH_SECRET`, `AUTH_COOKIE_SECURE`</code>
Auth email	<code>`AUTH_EMAIL_DELIVERY`, `AUTH_EMAIL_FROM`, `AUTH_EMAIL_WEBHOOK_URL`, `AUTH_EMAIL_WEBHOOK_SECRET`</code>
Government data	<code>`CONGRESS_API_KEY`, `CONGRESS_SYNC_CONGRESS`, `CONGRESS_SYNC_LIMIT`, `CONGRESS_SYNC_WRITE`</code>
Stripe billing	<code>`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, four `CAPITOL_LEDGER_STRIPE_*_PRICE_ID` values</code>
Weekly Brief	<code>`WEEKLY_BRIEF_DELIVERY`, `WEEKLY_BRIEF_FROM`, `WEEKLY_BRIEF_WEBHOOK_URL`, `WEEKLY_BRIEF_WEBHOOK_SECRET`, `WEEKLY_BRIEF_CRON_SECRET`</code>
Future hardening	<code>`UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN`, `OPENAI_API_KEY`, `SENTRY_DSN`</code>

Validation Commands

- ``pnpm production-auth:check`` confirms required production database tables and columns.
- ``pnpm production-auth:qa`` checks deployed auth behavior safely.
- ``pnpm auth-email:check`` checks verification and password-reset email provider readiness.
- ``pnpm weekly-brief:check`` checks Weekly Brief provider and scheduler settings.
- ``pnpm weekly-brief:qa`` checks the scheduled brief task route.
- ``pnpm billing:check`` checks Stripe/database readiness before live checkout testing.
- ``pnpm backend:check`` gives one consolidated outside-service setup snapshot.
- ``pnpm congress:check`` checks Congress.gov key, sync settings, and civic-data schema readiness.

What Can Wait

- RevenueCat or App Store subscription plumbing can wait until native iOS work begins.
- Sentry can wait until Vercel Observability is not enough.
- AI analysis should wait until official bill text, CRS summaries, votes, and source links are reliably synced.
- A separate job platform can wait until Vercel Cron is no longer enough.

Reference Links

- Vercel environment variables: <https://vercel.com/docs/environment-variables>
- Vercel cron jobs: <https://vercel.com/docs/cron-jobs/manage-cron-jobs>
- Neon Postgres: <https://neon.com/>
- Supabase backups: <https://supabase.com/docs/guides/platform/backups>
- Resend docs: <https://resend.com/docs>
- Postmark webhooks: <https://postmarkapp.com/developer/api/webhooks-api>
- Stripe Checkout Sessions: <https://docs.stripe.com/api/checkout/sessions>

- Stripe subscription webhooks: <https://docs.stripe.com/billing/subscriptions/webhooks>
- Congress.gov API: <https://api.congress.gov/>
- Upstash rate limiting: <https://upstash.com/docs/oss/sdks/ts/ratelimit/overview>
- OpenAI API pricing: <https://openai.com/api/pricing/>
- Vercel Observability: <https://vercel.com/docs/observability>